Milestone 3 — Hướng Dẫn & Tài Liệu Kỹ Thuật Tiền Xử Lý, Gán Nhãn & Tạo Đặc Trưng

> **Tài liệu kỹ thuật thực hiện các nhiệm vụ trong Milestone 3 (Reference Matching Pipeline)**
> Dành cho các công đoạn: Chuẩn hóa dữ liệu, Gán nhãn (Manual + Auto), Kỹ thuật đặc trưng (Feature Engineering), Sinh tập dữ liệu huấn luyện (Dataset Generation) và Phân chia tập (Splitting).

---

1. Tổng quan các nhiệm vụ đã hoàn thành

Trong Milestone 3, nhiệm vụ ghép nối tài liệu tham khảo (BibTeX entries trích xuất từ TeX $\leftrightarrow$ crawled references từ Semantic Scholar) bao gồm 6 nội dung trọng tâm:

1. Làm sạch và chuẩn hóa dữ liệu hai bảng `bibtex` và `references`.
2. Chuẩn bị công cụ/mẫu gán nhãn thủ công và tổng hợp kết quả của cả nhóm.
3. Viết thuật toán gán nhãn tự động bằng quy tắc heuristic và chuỗi tương đồng cho $\ge 10%$ số bài ngoài manual pool.
4. Thiết kế và trích xuất **14 đặc trưng (features)** phân biệt giữa 2 tài liệu (kèm lý do lý thuyết phục vụ Poster).
5. Sinh bảng dữ liệu cặp ứng viên kèm nhãn nhị phân ($0/1$).
6. Phân chia tập dữ liệu thành `train`, `valid`, `test` theo đúng quy chuẩn số lượng thành viên (k).
7. **Kết quả bàn giao:** Các file Parquet (`train_pairs.parquet`, `valid_pairs.parquet`, `test_pairs.parquet`) chứa đầy đủ vector đặc trưng và nhãn sẵn sàng cho bước huấn luyện mô hình.

---

2. Chi tiết 6 nhiệm vụ đã thực hiện

Mục 1: Làm sạch dữ liệu (`src/m3_matching/cleaning.py`)

Cung cấp các hàm chuẩn hóa nâng cao, **không sửa đổi bảng gốc**:

- **`normalize_title(title)`**:
  - Giải mã accent LaTeX trước khi bỏ ngoặc nhọn: `Schr\"{o}dinger` → `schrodinger` (không còn lỗi `schr o dinger`).
  - Loại bỏ các lệnh LaTeX phổ biến: `\textbf{...}`, `\emph{...}`, `\textit{...}`, `{...}`, math inline `$..$`.
  - Chuyển ký tự unicode có dấu về dạng ASCII không dấu (`_strip_accents`).
  - Chuyển chữ thường (lowercase), loại bỏ dấu câu thừa và chuẩn hóa khoảng trắng.
- **`normalize_authors(authors)`**:
  - Hỗ trợ cả hai dạng họ tên: `"Vaswani, Ashish"` và `"Ashish Vaswani"`.
  - Giữ tiền tố họ (`van`, `von`, `de`, `la`, `der`, ...) nhất quán giữa hai dạng: `"Jan van der Berg"` và `"van der Berg, Jan"` đều → `van der berg, j`.

- Loại bỏ các từ phụ: `"et al."`, `"others"`, `"and"`.
- Chuyển đổi về dạng chuẩn: `lastname, first_initial` (ví dụ: `vaswani, a`).

- **`normalize_year(value)`**:
  - Nhận diện năm công bố từ số nguyên, chuỗi `"2017"` hoặc chuỗi ngày ISO `"2017-06-12"`.
  - Kiểm tra tính hợp lệ trong khoảng $[1800, 2100]$, nếu lỗi trả về `None`.
- **`normalize_doi(value)`**:
  - Bỏ tiền tố `https://doi.org/`, `http://dx.doi.org/`, `doi:` và ký tự dư cuối chuỗi.
  - Lowercase để `https://doi.org/10.1/ABC` và `doi:10.1/abc` được coi là cùng DOI.
- **`prepare_bibtex_record(row)` & `prepare_reference_record(row)`**:
  - Bổ sung các trường chuẩn hóa: `title_norm`, `authors_norm`, `year_norm`, `doi_norm` mà giữ nguyên cấu trúc schema của dữ liệu gốc.
  - `prepare_reference_record` đọc DOI từ `doi` / `ref_doi` / `externalIds.DOI` (nếu nguồn M1 có).

Mục 2: Mẫu gán nhãn thủ công & Tổng hợp nhóm (`src/m3_matching/manual_tool.py`)

Quy định đề bài cho nhóm k thành viên:

- Số bài manual: $\ge 5 \times k$ bài (ví dụ nhóm 2 người cần ≥ 10 bài).
- Mỗi bài manual: ≥ 20 BibTeX entries được gán nhãn.
- Tổng số entries manual: $\ge 100 \times k$ entries.

Công cụ CLI hỗ trợ nhóm thực hiện (`manual_tool.py`):

1. **Tìm danh sách bài báo đủ điều kiện gán nhãn thủ công:**

```
python -m src.m3_matching.manual_tool --find-candidates
```

2. **Xuất file mẫu CSV kèm gợi ý candidate tự động cho 1 bài báo:**

```
python -m src.m3_matching.manual_tool --export-template 2310.12345
```

   *File xuất ra tại: `data/labels/manual_templates/2310.12345.csv`. File có sẵn cột `suggested_ref_title` và `suggested_score`, nhưng **không tự điền** `groundtruth_arxiv_id`. Thành viên mở Excel, đối chiếu gợi ý bằng mắt, tự điền `groundtruth_arxiv_id` (hoặc để trống nếu không khớp) rồi đổi `is_verified` thành `YES`.*
3. **Import kết quả sau khi các bạn trong nhóm đã kiểm tra:**

```
python -m src.m3_matching.manual_tool --import-csv data/labels/manual_templates/2310.12345.csv
```

   Mặc định chỉ import các dòng `is_verified=YES`; dòng chưa xác nhận bị bỏ qua để nhãn heuristic không lọt vào ground truth. Dùng `--allow-unverified` nếu thật sự cần import CSV cũ không có cột này. Dữ liệu được lưu chuẩn vào: `data/labels/{arxiv_id}.jsonl` với `"source": "manual"`.
4. **Kiểm tra tiến độ & Quota của cả nhóm:**

```
python -m src.m3_matching.manual_tool --summary --k 2 --n-publications 200
```

Mục 3: Gán nhãn tự động (`src/m3_matching/labelling.py`)

Quy định đề bài:

- Tự động gán nhãn cho $\ge 10%$ số bài báo ngoài nhóm manual (tối thiểu $\ge 2 \times k$ bài).
- Bài báo nào được chọn thì **bắt buộc phải gán nhãn cho 100% BibTeX entries** của bài đó.

Thuật toán Heuristic Matcher (`auto_match_one`): Ưu tiên độ chính xác tuyệt đối (High Precision) để ground truth không bị nhiễu:

1. **Quy tắc 1 (Exact DOI):** Khớp tuyệt đối nếu cả BibTeX và Reference đều có DOI trùng khớp.
2. **Quy tắc 2 (Exact Title):** Chuỗi tiêu đề sau chuẩn hóa trùng khớp 100% (độ dài ≥ 8 ký tự).
3. **Quy tắc 3 (Fuzzy Title + Author Overlap + Year):**
   - Tiêu đề có độ tương đồng sắp xếp từ (`rapidfuzz.token_sort_ratio >= 88%`).
   - Có ít nhất một tác giả trùng họ (hoặc Jaccard họ > 0).
   - Chênh lệch năm công bố ≤ 1 năm (hoặc thiếu thông tin năm).
4. **Kiểm tra xung đột (Ambiguity Check):** Nếu ứng viên thứ 2 có điểm số quá gần ứng viên tốt nhất (< 0.05) $\rightarrow$ bỏ qua không gán tự động.
5. Nếu không thỏa mãn bất kỳ quy tắc nào $\rightarrow$ gán `groundtruth_arxiv_id = None` (negative query).

Mục 4: Thiết kế 14 đặc trưng phân biệt (`src/m3_matching/features.py`)

Mỗi cặp ứng viên `(bibtex_entry, reference)` được mã hóa thành một vector 14 chiều. Bảng dưới đây liệt kê lý do và cơ sở lý thuyết (dùng để trình bày trên **Poster A2 & Báo cáo**):

STT	Tên đặc trưng (`feature_names`)	Giá trị	Ý tưởng thiết kế (Rationale)
1	`title_levenshtein`	`0.0 .. 1.0`	Khoảng cách chỉnh sửa ký tự chuẩn hóa (Levenshtein) giữa 2 tiêu đề.
2	`title_token_sort`	`0.0 .. 1.0`	Đo độ tương đồng sau khi sắp xếp từ (Token Sort Ratio qua RapidFuzz), xử lý đảo từ và dấu câu.
3	`title_token_set`	`0.0 .. 1.0`	Đo độ tương đồng bao hàm tập từ (Token Set Ratio), xử lý tiêu đề phụ (subtitle) hoặc phần mở rộng.
4	`title_jaccard`	`0.0 .. 1.0`	Tỷ lệ giao trên hợp của tập từ vựng xuất hiện trong 2 tiêu đề.
5	`title_exact`	`0.0` / `1.0`	Cờ nhị phân: 1.0 nếu tiêu đề sau khi làm sạch trùng nhau hoàn toàn.
6	`title_len_diff_ratio`	`0.0 .. 1.0`	Độ lệch độ dài tương đối: $\|len_1 - len_2\| / \max(len_1, len_2)$.
7	`author_jaccard`	`0.0 .. 1.0`	Jaccard similarity trên tập các **họ** (last names) của các tác giả.
8	`author_first_match`	`0.0` / `1.0`	Cờ nhị phân: 1.0 nếu tác giả đầu tiên (tác giả chính) trùng họ.
9	`author_any_match`	`0.0` / `1.0`	Cờ nhị phân: 1.0 nếu có ít nhất một tác giả trùng họ giữa 2 tài liệu.
10	`year_diff`	Số thực	Hiệu số năm công bố tuyệt đối $\|y_1 - y_2\|$. Nếu thiếu năm gán sentinel `99.0`.
11	`year_exact`	`-1 / 0 / 1`	1.0 nếu trùng năm, 0.0 nếu khác năm, -1.0 nếu một bên thiếu năm.
12	`year_missing`	`0.0` / `1.0`	1.0 nếu một trong hai tài liệu bị khuyết thông tin năm.
13	`doi_match`	`-1 / 0 / 1`	1.0 nếu DOI trùng khớp (sau chuẩn hóa tiền tố), -1.0 nếu cả 2 có DOI nhưng khác nhau, 0.0 nếu thiếu DOI.
14	`cite_count`	Số thực	Tần suất xuất hiện của `citation_key` trong văn bản bài báo (tận dụng từ M2 hierarchy). Mặc định `0.0` nếu chưa truyền `extra` context.

> `pair_feature_vector` nhận tham số `extra` tùy chọn cho `cite_count`; các đặc trưng còn lại được tính trực tiếp từ cặp `(bib, ref)`. `generate_training_dataset` tự đọc `elements.jsonl` (M2) nếu có trong `TABLES_DIR` và đếm các lệnh `\cite{...}` để truyền `cite_count` vào từng cặp. Nếu bảng `elements` chưa được cung cấp, `cite_count = 0.0` cho mọi cặp (được ghi rõ là sentinel, không phải giá trị thật).

Mục 5: Tạo bảng dữ liệu huấn luyện (`src/m3_matching/dataset.py`)

Hàm `generate_training_dataset(...)`:

- Với mỗi bài báo, tạo tích Descartes giữa các BibTeX entries đã gán nhãn và các crawled references có arXiv ID.
- Dedup theo `ref_arxiv_id` trong cùng publication để reference bị crawl trùng không sinh cặp nhân đôi.
- Tính toán vector 14 đặc trưng cho từng cặp; `cite_count` lấy từ `elements.jsonl` khi có (xem Mục 4).
- Gán nhãn học máy:
  - `label = 1` nếu `ref_arxiv_id == groundtruth_arxiv_id` (Positive pair).
  - `label = 0` nếu `ref_arxiv_id != groundtruth_arxiv_id` hoặc `groundtruth == None` (Negative pair).
- Lưu kết quả ra định dạng Parquet sharded chuẩn cho bước huấn luyện mô hình.

Mục 6: Phân chia tập dữ liệu (`src/m3_matching/split.py`)

Quy tắc phân chia nghiêm ngặt theo **Publication**:

- **Không bao giờ chia theo từng cặp hoặc từng entry** (tránh hiện tượng Data Leakage giữa các tập).
- Hỗ trợ số lượng thành viên nhóm k:
  - Tập **Test**: Đúng k bài manual + k bài auto (nhóm 2 người là 2 manual + 2 auto).
  - Tập **Valid**: Đúng k bài manual + k bài auto.
  - Tập **Train**: Toàn bộ các bài báo đã gán nhãn còn lại.
- Sử dụng tham số `seed` ngẫu nhiên cố định (mặc định `42`) để đảm bảo tính tái lập (reproducible).

---

3. Hướng dẫn chạy trọn gói (`prepare_dataset.py`)

Chỉ cần thực thi một lệnh duy nhất để tự động thực hiện toàn bộ quy trình:

```
# Chạy thực tế cho nhóm 2 người:
python -m src.m3_matching.prepare_dataset --k 2

# Giới hạn số bài auto để tránh dataset quá lớn (ví dụ k=1):
python -m src.m3_matching.prepare_dataset --k 1 --max-auto 450

# Hoặc chạy thử nghiệm ngay (chế độ demo dữ liệu mẫu):
python -m src.m3_matching.prepare_dataset --demo
```

> Trên Windows console (cp1258), script đã tự ép UTF-8 để in tiếng Việt. Nếu vẫn lỗi, chạy thêm `$env:PYTHONUTF8=1` trước lệnh. Input thật cần có `data/tables/bibtex.jsonl` và `data/tables/references.jsonl` (từ M1/M2); nhãn thủ công đặt tại `data/labels/{arxiv_id}.jsonl`.

Các file xuất ra sẵn sàng tại `data/tables/`:

- `data/tables/train_pairs.parquet` $\rightarrow$ Dữ liệu dùng để huấn luyện mô hình (Training set).
- `data/tables/valid_pairs.parquet` $\rightarrow$ Dữ liệu dùng để điều chỉnh siêu tham số và ngưỡng (Validation set).

- `data/tables/test_pairs.parquet` $\rightarrow$ Dữ liệu dùng để đánh giá MRR@5 cuối cùng (Test set).
- `data/tables/dataset_pairs.parquet` $\rightarrow$ Toàn bộ dữ liệu cặp tổng hợp.

---

4. Hướng dẫn nạp dữ liệu huấn luyện mô hình

Đoạn code mẫu nạp dữ liệu cặp đã xử lý:

```
import pandas as pd
from src.m3_matching.features import feature_names

# 1. Đọc dữ liệu cặp đã chuẩn bị sẵn
train_df = pd.read_parquet("data/tables/train_pairs.parquet")
valid_df = pd.read_parquet("data/tables/valid_pairs.parquet")

# 2. Lấy 14 đặc trưng và nhãn
features = feature_names()
X_train, y_train = train_df[features], train_df["label"]
X_valid, y_valid = valid_df[features], valid_df["label"]

# Dữ liệu đã sẵn sàng để đưa vào huấn luyện mô hình (ví dụ: RandomForest, LightGBM, ...)
```

---

5. Kết quả kiểm thử Unit Test

Đã tạo bộ kiểm thử riêng độc lập tại `tests/test_m3_matching.py`. Chạy kiểm tra:

```
python -m pytest tests/test_m3_matching.py -v
```

Kết quả: **21/21 test cases passed (100%)**, bao phủ làm sạch (title/author/year/DOI, accent LaTeX, tiền tố họ), gán nhãn thủ công & auto, 14 đặc trưng, dedup reference, `cite_count`, sinh bảng cặp và chia tập (kể cả kiểm tra `k` không hợp lệ).

---

6. Chạy trên dữ liệu thật M1/M2

6.1. Chuẩn bị hai bảng nội bộ

`data/tables/bibtex.jsonl` lấy từ output M2:

```
M2_range_1_tables.zip
└── bibtex.jsonl
```

`data/tables/references.jsonl` ghép từ các file `references_s2.json` của M1 (mỗi file là JSON Lines, mỗi dòng là một reference của paper đó):

```
from pathlib import Path

src = Path("references")          # thư mục references của M1
out = Path("data/tables/references.jsonl")
out.parent.mkdir(parents=True, exist_ok=True)
with out.open("w", encoding="utf-8") as fh:
    for paper_dir in sorted(src.iterdir()):
        f = paper_dir / "references_s2.json"
        if f.exists():
            for line in f.read_text(encoding="utf-8").splitlines():
                if line.strip():
                    fh.write(line.strip() + "\n")
```

Chỉ các publication có **ít nhất một reference mang arXiv ID** mới vào pool ghép cặp; publication không có ứng viên bị loại tự động.

6.2. Tạo mẫu gán nhãn thủ công

```
python -m src.m3_matching.manual_tool --find-candidates
python -m src.m3_matching.manual_tool --export-template 2609.38024
python -m src.m3_matching.manual_tool --import-csv data/labels/manual_templates/2609.38024.csv
```

Lặp lại cho đủ `5 * k` bài. Mỗi CSV có sẵn `suggested_ref_arxiv_id` và `suggested_score` để tham khảo; `--export-template` **không tự điền** `groundtruth_arxiv_id`. Nhóm phải kiểm tra bằng mắt, điền groundtruth (hoặc để trống nếu không khớp) và đổi `is_verified` thành `YES` trước khi import. Nếu chưa có dòng nào `is_verified=YES`, lệnh import sẽ báo lỗi thay vì ghi nhãn rỗng.

6.3. Chạy pipeline và kết quả bàn giao

```
python -m src.m3_matching.prepare_dataset --k 1 --max-auto 450
```

Kết quả chạy thật trên `M2_range_1_tables.zip` + references M1 (k=1):

- 5 bài manual × 45 entries = 225 entries; 450 bài auto.
- `train_pairs.parquet`: 26 017 cặp (1 151 positive / 24 866 negative), 451 bài.
- `valid_pairs.parquet`: 1 753 cặp (44 positive / 1 709 negative), 2 bài.
- `test_pairs.parquet`: 1 419 cặp (33 positive / 1 386 negative), 2 bài.
- `dataset_pairs.parquet`: 29 189 cặp, 455 bài, đủ 14 cột đặc trưng.

> **Lưu ý:** các nhãn manual đang có trong lần chạy này được tạo trước khi bản sửa đổi này (gợi ý heuristic score ≥ 0.85) để pipeline chạy end-to-end. Từ bản hiện tại, `--export-template` không tự điền

> `groundtruth_arxiv_id`. Trước khi nộp, nhóm vẫn cần mở `data/labels/manual_templates/`, kiểm tra bằng mắt, điền groundtruth + `is_verified=YES`, rồi import lại CSV.